

上海电力大学

数据库应用课程设计



题 目：易宿酒店预订管理系统数据库设计与实现

学生姓名：_

学 号：_

院 系：_ 计算机科学与技术学院 _

专业班级：_

年 月 日

一、需求分析

1.1 问题描述

本课程设计以“易宿酒店预订管理系统”为对象，围绕酒店浏览、客房预订、入住人登记、订单退订、评价反馈和后台统计等业务建立数据库。系统面向普通用户和管理员两类角色：普通用户完成酒店查询、订单创建、订单查询和评价；管理员维护酒店、房型、订单状态和统计报表。数据库需要保存用户、入住人、酒店、房型、订单、订单明细、入住人关联和评价等数据，并通过约束、索引、视图、存储过程和触发器保证数据一致性。

系统设计重点不是简单保存页面数据，而是把酒店预订业务中“房型库存、订单状态、入住人画像、价格统计和权限控制”等数据库问题明确建模。用户下单时，订单主表记录整体订单信息，订单明细表记录每一间客房，入住人关联表记录每间房对应的入住人；订单退订时，数据库需要恢复对应房型的可预订数量。

1.2 系统功能描述

用户管理：保存用户账号、密码密文、手机号、身份证号、积分、会员等级和角色信息，支持普通用户和管理员两种权限。

入住人管理：用户维护常用入住人，包括姓名、身份证号、性别、出生日期、职业、受教育程度和收入状况，相关字段用于画像统计。

酒店与房型管理：管理员维护酒店名称、城市、区域、地址、星级、联系电话及房型价格、容量、总数量和可预订数量。

酒店检索：用户按城市、区域、酒店名称和价格区间组合查询酒店及房型，查询结果来自酒店房型视图。

客房预订：用户选择酒店、房型、入住日期、离店日期和入住人后生成订单，系统校验日期、库存、容量和价格。

订单退订与评价：用户可退订未入住订单，完成订单可评价一次；评价表通过唯一约束保证一单一评。

统计报表：管理员按城市、区域、酒店、价格区间、入住人画像等维度统计订单数量、间夜数、金额和入住率。

图1 系统业务流程图

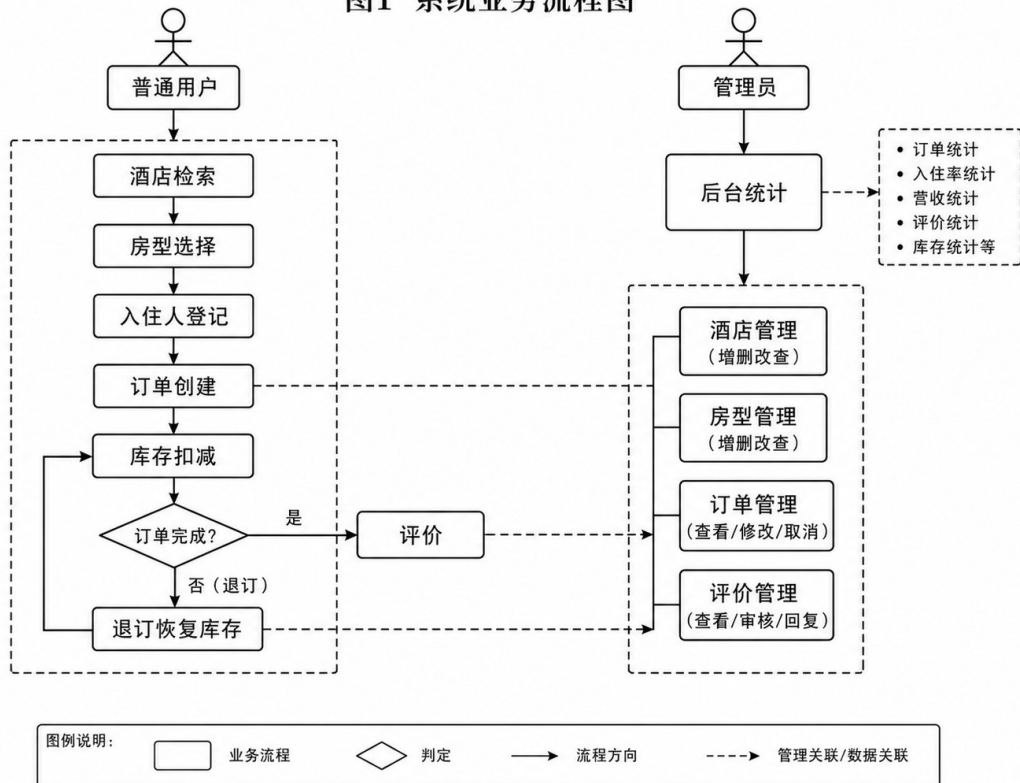


图 1 系统业务流程图

1.3 数据库完整性要求

实体完整性方面，所有业务表均使用自增主键或联合主键，例如 `t_user.user_id`、`t_hotel.hotel_id`、`t_order.order_id`、`t_item_guest(item_id, guest_id)`。参照完整性方面，入住人必须属于某个用户，房型必须属于某个酒店，订单必须关联用户和酒店，订单明细必须关联订单和房型，评价必须关联订单和用户。

用户定义完整性方面，系统设置多项 CHECK 约束：身份证号为空或长度为 18 位，积分不能为负数，会员等级只能为“普通、VIP、SVIP”，角色只能为“用户、管

理员”，酒店星级在 1 到 5 之间，房价不能小于 0，房型可订数量必须在 0 到总数量之间，离店日期必须晚于入住日期，评价分数必须在 1 到 5 之间。

触发器完整性方面，插入订单明细后自动扣减房型 `available_count`；订单从“已预订”变为“已退订”时自动恢复对应房型库存。存储过程方面，系统提供用户订单查询、入住率统计、预订情况统计和入住人画像统计等过程，减少应用层重复拼接复杂统计 SQL。

1.4 社会、安全、法律因素

本系统保存手机号、身份证号、出生日期和收入状况等敏感信息，数据库设计需要遵循最小化采集和按角色授权原则。普通用户只能访问自己的订单和入住人信息，管理员只能在管理业务范围内查看用户列表和统计数据。密码不以明文保存，应用层使用 BCrypt 加密后写入数据库，数据库中只保存密文。

从安全角度看，系统所有查询均应使用参数绑定，不允许拼接外部输入形成 SQL。统计和查询场景通过视图、存储过程和固定参数控制访问路径，减少越权查询和 SQL 注入风险。从法律角度看，身份证号、手机号等个人信息只用于订房和入住登记，不应在统计报表中直接展示明文，应优先使用聚合结果或脱敏展示。

二、概念结构设计

根据需求分析，本系统抽象出用户、入住人、酒店、房型、订单、订单明细、入住人关联和评价八类核心实体。用户与入住人为一对多关系，用户与订单为一对多关系，酒店与房型为一对多关系，酒店与订单为一对多关系，订单与订单明细为一对多关系，订单明细与入住人为多对多关系并通过 t_item_guest 关联表实现，订单与评价为一对一关系。

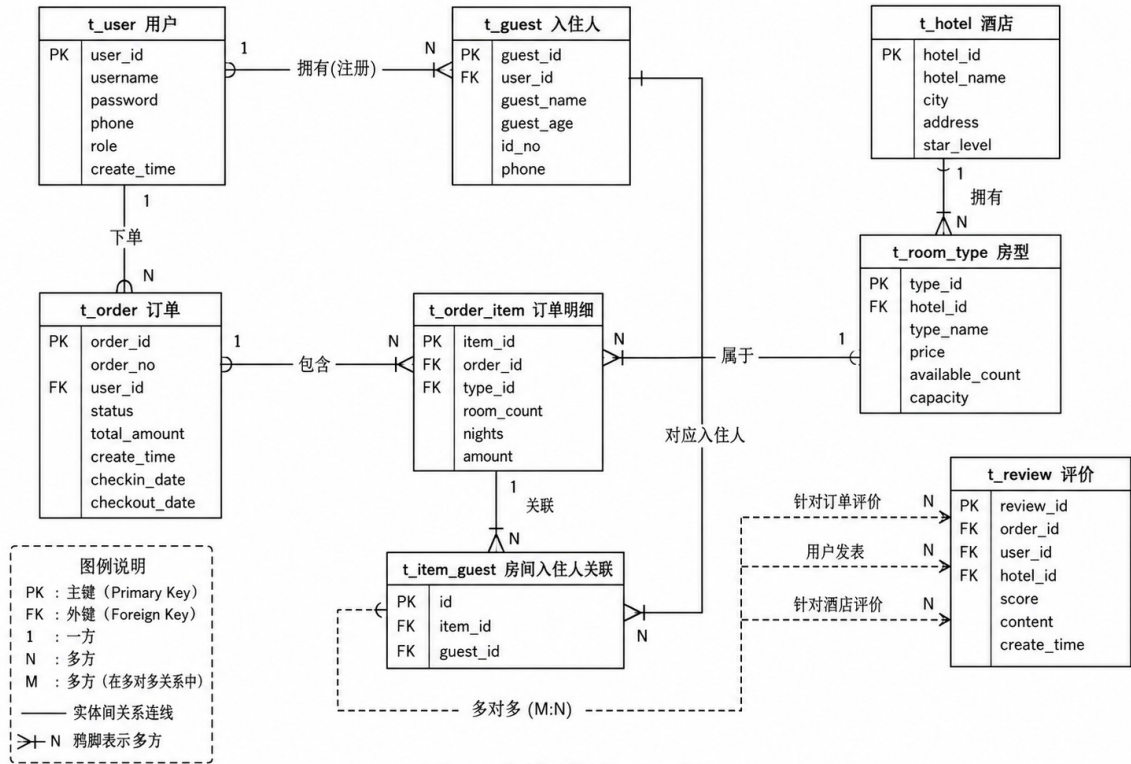


图2 系统整体 E-R 图

图 2 系统整体 E-R 图

用户实体包含 user_id、username、password、phone、id_card、points、level、role、created_at 等属性，其中 username 唯一，password 保存 BCrypt 密文。入住人实体包含 guest_id、user_id、name、id_card、

gender、birth_date、occupation、education、income_level 等属性，用于订房登记和画像统计。

酒店实体包含 hotel_id、hotel_name、city、district、address、star、phone。房型实体包含 room_type_id、hotel_id、type_name、price、capacity、total_count、available_count，其中 available_count 是随订单变化的库存字段，由触发器维护。订单实体包含 order_id、order_no、user_id、hotel_id、check_in_date、check_out_date、total_price、pay_price、status、created_at。

订单明细实体表示“订单中的一间房”，包含 item_id、order_id、room_type_id、price。评价实体包含 review_id、order_id、user_id、score、content、created_at，通过 order_id 唯一约束保证每个订单最多评价一次。

三、逻辑结构设计

3.1 关系模式设计

表 1 核心关系模式汇总

关系名	主要属性	主码
t_user	user_id, username, password, phone, id_card, points, level, role, created_at	user_id
t_guest	guest_id, user_id, name,	guest_id

	id_card, gender, birth_date, occupation, education, income_level	
t_hotel	hotel_id, hotel_name, city, district, address, star, phone	hotel_id
t_room_type	room_type_id, hotel_id, type_name, price, capacity, total_count, available_count	room_type_id
t_order	order_id, order_no, user_id, hotel_id, check_in_date, check_out_date, total_price, pay_price, status, created_at	order_id
t_order_item	item_id, order_id, room_type_id, price	item_id
t_item_guest	item_id, guest_id	item_id + guest_id
t_review	review_id, order_id, user_id, score, content, created_at	review_id

3.2 数据字典

表 2 t_user 用户表数据字典

字段	类型	约束/说明
user_id	INT	主键，自增
username	VARCHAR(20)	非空，唯一
password	VARCHAR(64)	非空，BCrypt 密文
phone	CHAR(11)	非空
id_card	CHAR(18)	身份证号，可空但长度为 18
points	INT	默认 0，非负
level	VARCHAR(10)	普通/VIP/SVIP

role	VARCHAR(10)	用户/管理员
------	-------------	--------

表 3 t_hotel 与 t_room_type 数据字典

表名	字段	类型与约束
t_hotel	hotel_id	INT 主键，自增
t_hotel	hotel_name	VARCHAR(50) 非空
t_hotel	city, district	VARCHAR(20) 非空，用于区域 查询
t_hotel	star	TINYINT, 1 到 5 星
t_room_type	room_type_id	INT 主键，自增
t_room_type	hotel_id	INT 外键，关联酒店
t_room_type	price	DECIMAL(10,2), 非负
t_room_type	available_count	INT, 0 到 total_count, 由触 发器维护

表 4 t_order 与 t_order_item 数据字典

表名	字段	类型与约束
t_order	order_no	VARCHAR(32) 非空，唯一
t_order	user_id, hotel_id	INT 外键，关联用户和酒店
t_order	check_in_date	DATE 非空，默认当前日期

t_order	check_out_date	DATE 非空，必须晚于入住日期
t_order	status	已预订/已入住/已完成/已退订
t_order_item	item_id	INT 主键，自增
t_order_item	room_type_id	INT 外键，关联房型
t_order_item	price	DECIMAL(10,2)，下单时房价 快照

3.3 子模式设计

系统定义 v_hotel_room 视图用于酒店检索和房型展示。该视图将 t_hotel 与 t_room_type 连接，输出酒店名称、城市、区域、地址、星级、房型、价格、可住人数和可订数量。前端查询酒店时不需要直接访问多张表，只需要按照城市、区域和价格条件查询该视图。

系统定义 v_order_detail 视图用于订单详情和统计报表。该视图将订单、用户、酒店、订单明细、房型、入住人关联和入住人画像连接起来，输出订单号、用户名、订单状态、入住离店日期、酒店城市、房型、房价、入住人姓名、性别、年龄、职业、教育和收入状况。

代码 1 视图 v_hotel_room 定义

```
CREATE OR REPLACE VIEW v_hotel_room AS
SELECT h.hotel_id, h.hotel_name, h.city, h.district, h.address, h.star,
```

```
        rt.room_type_id, rt.type_name, rt.price, rt.capacity,  
        rt.total_count, rt.available_count  
FROM t_hotel h  
JOIN t_room_type rt ON rt.hotel_id = h.hotel_id;
```

四、数据库的物理设计

4.1 索引设计

酒店检索和统计报表是本系统访问频率最高的两类操作。酒店检索通常按城市、区域、酒店名和价格范围组合查询，因此在 `t_hotel(city, district)`、`t_hotel(hotel_name)`、`t_room_type(hotel_id)` 上建立索引。订单查询通常按用户和状态过滤，因此在 `t_order(user_id, status)` 上建立联合索引。统计报表经常按入住日期筛选，因此在 `t_order(check_in_date)` 上建立索引。

代码 2 索引创建 SQL

```
CREATE INDEX idx_hotel_city_district ON t_hotel (city, district);  
CREATE INDEX idx_hotel_name          ON t_hotel (hotel_name);  
CREATE INDEX idx_room_hotel          ON t_room_type (hotel_id);  
CREATE INDEX idx_order_user_status   ON t_order (user_id, status);  
CREATE INDEX idx_order_checkin       ON t_order (check_in_date);
```

`idx_hotel_city_district` 能够支持“城市 + 区域”的等值查询，减少酒店检索时扫描的行数。`idx_room_hotel` 能够支持查询某个酒店下所有房型。`idx_order_user_status` 能够支持用户进入“我的订单”页面时按订单状态筛选。

idx_order_checkin 能够支持统计某个时间段内订单数量、间夜数和金额。

4.2 性能优化策略

除索引外，系统还采用视图复用复杂连接，避免应用层重复编写多表 JOIN。统计类 SQL 统一封装为存储过程，应用层只传入时间范围和统计维度，数据库内部完成聚合计算。对于订单明细和入住人画像统计，系统采用订单详情视图作为查询基础，使业务代码更简单。

库存字段 available_count 属于冗余字段，但它可以显著降低订房时查询剩余房量的成本。为了避免冗余字段与订单明细不一致，数据库通过触发器在订单明细插入和订单退订时自动维护库存。该设计用触发器换取查询效率，同时保证一致性。

五、数据库设计实现及运行

5.1 数据库创建

代码 3 创建数据库 SQL

```
DROP DATABASE IF EXISTS hotel_booking;
CREATE DATABASE hotel_booking DEFAULT CHARACTER SET utf8mb4;
USE hotel_booking;
```

5.2 数据表创建与完整性约束

代码 4 用户表与入住人表建表 SQL

```
CREATE TABLE t_user (
```

```

user_id    INT PRIMARY KEY AUTO_INCREMENT,
username   VARCHAR(20) NOT NULL UNIQUE,
password   VARCHAR(64) NOT NULL,
phone      CHAR(11)    NOT NULL,
id_card    CHAR(18),
points     INT          DEFAULT 0,
level      VARCHAR(10) DEFAULT '普通',
role       VARCHAR(10) DEFAULT '用户',
created_at DATETIME     DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT ck_user_idcard CHECK (id_card IS NULL OR CHAR_LENGTH(id_card) =
18),
CONSTRAINT ck_user_points CHECK (points >= 0),
CONSTRAINT ck_user_level  CHECK (level IN ('普通', 'VIP', 'SVIP')),
CONSTRAINT ck_user_role   CHECK (role IN ('用户', '管理员'))
);

CREATE TABLE t_guest (
    guest_id    INT PRIMARY KEY AUTO_INCREMENT,
    user_id     INT NOT NULL,
    name        VARCHAR(20) NOT NULL,
    id_card     CHAR(18),
    gender      CHAR(1) DEFAULT '男',
    birth_date  DATE,
    occupation  VARCHAR(20),
    education   VARCHAR(10),
    income_level VARCHAR(10),
    CONSTRAINT fk_guest_user FOREIGN KEY (user_id) REFERENCES t_user (user_id),
    CONSTRAINT ck_guest_gender CHECK (gender IN ('男', '女'))
);

```

代码 5 酒店、房型和订单核心建表 SQL

```

CREATE TABLE t_hotel (
    hotel_id    INT PRIMARY KEY AUTO_INCREMENT,
    hotel_name  VARCHAR(50) NOT NULL,
    city        VARCHAR(20) NOT NULL,
    district    VARCHAR(20) NOT NULL,
    address     VARCHAR(100),
    star        TINYINT,
    phone       VARCHAR(20),
    CONSTRAINT ck_hotel_star CHECK (star BETWEEN 1 AND 5)
);

```

```
);

CREATE TABLE t_room_type (
    room_type_id    INT PRIMARY KEY AUTO_INCREMENT,
    hotel_id        INT NOT NULL,
    type_name       VARCHAR(20) NOT NULL,
    price           DECIMAL(10,2) DEFAULT 0,
    capacity        TINYINT NOT NULL,
    total_count     INT NOT NULL,
    available_count INT NOT NULL,
    CONSTRAINT fk_room_hotel FOREIGN KEY (hotel_id) REFERENCES t_hotel
(hotel_id),
    CONSTRAINT ck_room_price CHECK (price >= 0),
    CONSTRAINT ck_room_avail CHECK (available_count BETWEEN 0 AND total_count)
);
```

5.3 视图创建与测试

代码 6 订单详情视图 SQL

```
CREATE OR REPLACE VIEW v_order_detail AS
SELECT o.order_id, o.order_no, u.username, o.status,
       o.check_in_date, o.check_out_date, h.hotel_name, h.city, h.district,
       rt.type_name, oi.price AS room_price,
       g.name AS guest_name, g.gender,
       TIMESTAMPDIFF(YEAR, g.birth_date, CURDATE()) AS guest_age,
       g.occupation, g.education, g.income_level
FROM t_order o
JOIN t_user u ON u.user_id = o.user_id
JOIN t_hotel h ON h.hotel_id = o.hotel_id
LEFT JOIN t_order_item oi ON oi.order_id = o.order_id
LEFT JOIN t_room_type rt ON rt.room_type_id = oi.room_type_id
LEFT JOIN t_item_guest ig ON ig.item_id = oi.item_id
LEFT JOIN t_guest g ON g.guest_id = ig.guest_id;
```

视图查询测试结果

SELECT order_no, username, status, hotel_name, type_name, guest_name, guest_age FROM v_order_detail LIMIT 5;						
order_no	username	status	hotel_name	type_name	guest_name	guest_age
H820260605001	zhangsan	已完成	北京中关村智选酒	标准间	张三	28
H820260610001	lisi	已完成	上海外滩景观酒店	江景大床房	李四	41
H820260628001	wangwu	已入住	北京国贸大酒店	豪华套房	王五	55
H820260702001	zhangsan	已预订	杭州西湖山水酒店	湖景大床房	王丽	27

图 3 视图查询测试结果截图

5.4 存储过程定义与测试

代码 7 用户订单查询存储过程

```
CREATE PROCEDURE sp_user_orders(IN p_user_id INT)
BEGIN
    SELECT o.order_id, o.order_no, h.hotel_name, h.city, h.district,
           o.check_in_date, o.check_out_date, o.total_price,
           o.pay_price, o.status, o.created_at
    FROM t_order o
    JOIN t_hotel h ON h.hotel_id = o.hotel_id
    WHERE o.user_id = p_user_id
    ORDER BY o.created_at DESC;
END;
```

代码 8 入住率统计存储过程核心 SQL

```
CREATE PROCEDURE sp_occupancy_rate(IN p_start DATE, IN p_end DATE)
BEGIN
    DECLARE v_days INT DEFAULT DATEDIFF(p_end, p_start) + 1;
    SELECT h.hotel_name, h.city, rt.type_name, rt.total_count,
           COALESCE(SUM(GREATEST(0, DATEDIFF(
               LEAST(o.check_out_date, DATE_ADD(p_end, INTERVAL 1 DAY)),
               GREATEST(o.check_in_date, p_start)))), 0) AS sold_nights,
           rt.total_count * v_days AS total_nights
```

```
FROM t_room_type rt
JOIN t_hotel h ON h.hotel_id = rt.hotel_id
LEFT JOIN t_order_item oi ON oi.room_type_id = rt.room_type_id
LEFT JOIN t_order o ON o.order_id = oi.order_id
    AND o.status <> '已退订'
    AND o.check_in_date <= p_end
    AND o.check_out_date > p_start
    GROUP BY rt.room_type_id, h.hotel_name, h.city, rt.type_name,
rt.total_count;
END;
```

存储过程统计结果

CALL sp_occupancy_rate("2026-06-01", "2026-07-31");

hotel_name	city	type_name	sold_nights	total_nights	occupancy_rate
上海外滩景观酒店	上海	江景大床房	10	54	18.52%
北京国贸大酒店	北京	豪华套房	6	155	3.87%
杭州西湖山水酒店	杭州	湖景大床房	4	496	0.81%

图 4 存储过程统计结果截图

5.5 触发器定义与测试

代码 9 库存扣减与退订恢复触发器

```
CREATE TRIGGER trg_order_item_insert
AFTER INSERT ON t_order_item
FOR EACH ROW
BEGIN
    UPDATE t_room_type
    SET available_count = available_count - 1
    WHERE room_type_id = NEW.room_type_id;
END;

CREATE TRIGGER trg_order_cancel
AFTER UPDATE ON t_order
FOR EACH ROW
```



```
BEGIN
  IF OLD.status = '已预订' AND NEW.status = '已退订' THEN
    UPDATE t_room_type rt
    JOIN (
      SELECT room_type_id, COUNT(*) AS cnt
      FROM t_order_item
      WHERE order_id = NEW.order_id
      GROUP BY room_type_id
    ) x ON x.room_type_id = rt.room_type_id
    SET rt.available_count = rt.available_count + x.cnt;
  END IF;
END;
```

触发器测试步骤为：先查询某一房型的 available_count；插入订单明细后再次查询，结果应减少 1；将订单状态从“已预订”更新为“已退订”后再次查询，结果应恢复。该测试说明数据库层能够自动维护库存，不依赖前端或后端遗漏风险较高的手动更新。

触发器库存变化测试

SELECT room_type_id, available_count FROM t_room_type WHERE room_type_id = 4;		
步骤	SQL 操作	available_count
1	预订前查询房型库存	30
2	插入 t_order_item 订单明细	29
3	订单状态更新为已退订	30

图 5 触发器库存变化测试截图

5.6 数据操纵语句

代码 10 插入、删除、修改和查询操作示例

```
-- 插入酒店
INSERT INTO t_hotel (hotel_name, city, district, address, star, phone)
VALUES ('上海浦东商务酒店', '上海', '浦东新区', '世纪大道100号', 4, '021-68881234');

-- 查询酒店房型
SELECT * FROM v_hotel_room
WHERE city = '上海' AND price BETWEEN 300 AND 900
ORDER BY price ASC;

-- 修改订单状态
UPDATE t_order SET status = '已退订'
WHERE order_id = 5 AND status = '已预订';

-- 删除未被订单引用的房型
DELETE FROM t_room_type
WHERE room_type_id = 15
AND NOT EXISTS (SELECT 1 FROM t_order_item WHERE room_type_id = 15);
```

5.7 原子事务与并发控制

客房预订属于需要原子性保证的操作。一次预订至少涉及订单主表、订单明细表和房间入住人关联表，任何一步失败都不能留下半完成订单。因此应用层在创建订单时应显式开启事务，数据库层通过外键和触发器进一步保证明细和库存一致。

对于高并发订房场景，可以在扣减库存前通过条件更新判断 `available_count` 是否大于 0，也可以使用事务隔离级别或行级锁减少超卖风险。

代码 11 预订事务伪代码

```
START TRANSACTION;
INSERT INTO t_order (...);
INSERT INTO t_order_item (order_id, room_type_id, price) VALUES (...);
```

```
INSERT INTO t_item_guest (item_id, guest_id) VALUES (...);  
-- 插入订单明细后触发器自动扣减库存  
COMMIT;  
-- 任意一步失败则 ROLLBACK
```

六、系统详细设计及实现

6.1 系统功能模块说明

系统从数据库角度划分为用户与权限模块、酒店房型模块、订单预订模块、评价模块和统计报表模块。用户与权限模块负责用户账号、入住人、角色和密码密文；酒店房型模块负责酒店、房型和库存；订单预订模块负责订单主表、订单明细和房间入住人关联；评价模块负责订单完成后的评分和文字评价；统计报表模块通过视图和存储过程输出预订量、入住率和入住人画像。

图6 易宿酒店预订管理系统功能模块图

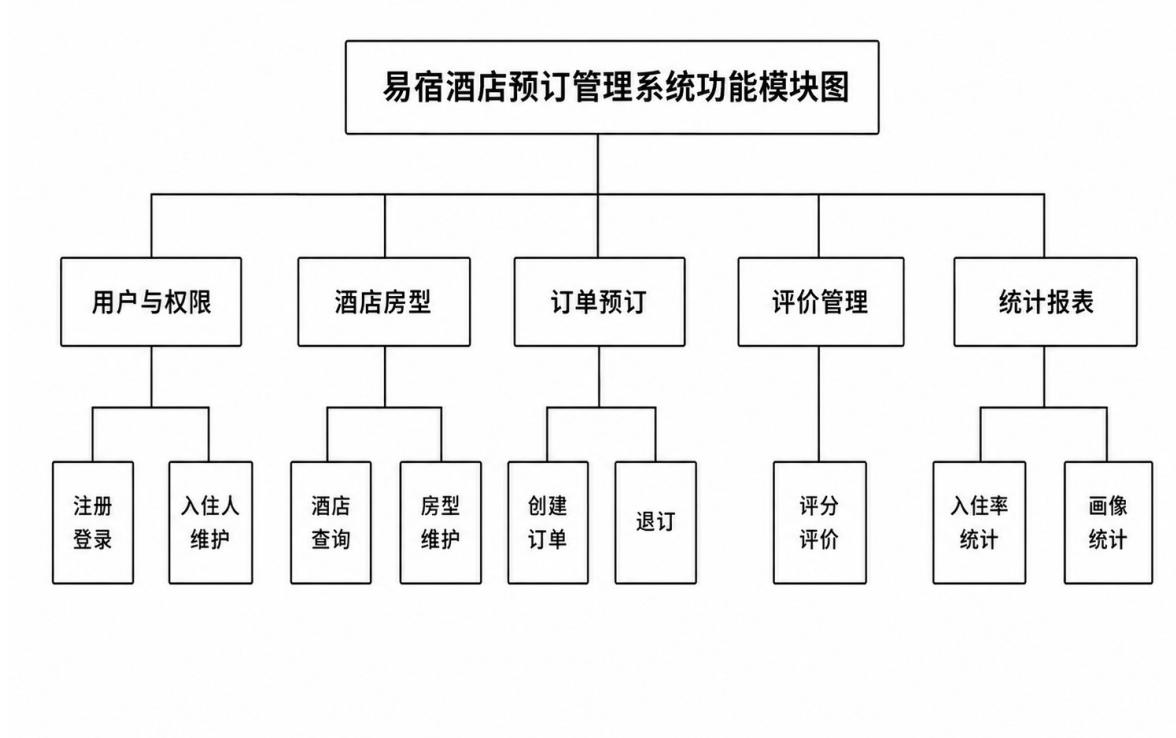


图 6 系统功能模块图

6.2 数据库增删改查关键代码

代码 12 酒店组合查询 Mapper SQL

```
SELECT h.*, MIN(rt.price) AS min_price
FROM t_hotel h
JOIN t_room_type rt ON rt.hotel_id = h.hotel_id
WHERE (:city IS NULL OR h.city = :city)
      AND (:district IS NULL OR h.district = :district)
      AND (:name IS NULL OR h.hotel_name LIKE CONCAT('%', :name, '%'))
      AND (:priceMin IS NULL OR rt.price >= :priceMin)
      AND (:priceMax IS NULL OR rt.price <= :priceMax)
GROUP BY h.hotel_id
ORDER BY min_price ASC;
```

该查询体现了多条件组合检索。城市、区域、酒店名和价格范围均为可选条件，

数据库通过参数绑定接收外部输入，不在 SQL 字符串中直接拼接用户输入，从而降低 SQL 注入风险。

代码 13 调用存储过程的后端代码

```
public List<Map<String, Object>> occupancyRate(LocalDate start, LocalDate end)
{
    Assert.notNull(start, "开始日期不能为空");
    Assert.notNull(end, "结束日期不能为空");
    Assert.isTrue(!end.isBefore(start), "结束日期不能早于开始日期");
    return statsMapper.occupancyRate(start, end);
}

@Select("{CALL sp_occupancy_rate(#{start}, #{end})}")
List<Map<String, Object>> occupancyRate(LocalDate start, LocalDate end);
```

统计服务只负责校验参数和调用 Mapper，具体聚合逻辑由数据库存储过程完成。这样可以把复杂统计 SQL 固化在数据库层，避免多个接口重复维护同一段统计逻辑。

系统界面与数据库结果对照图

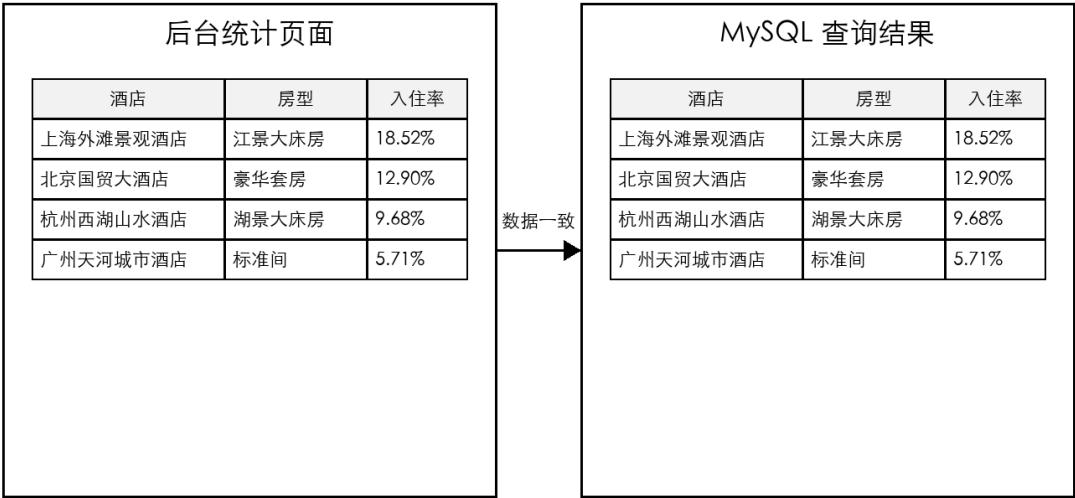


图 7 系统界面与数据库结果对照截图

七、总结

本次数据库应用课程设计围绕酒店预订业务完成了从需求分析、概念结构设计、逻辑结构设计、物理设计到数据库实现与运行测试的完整过程。系统在数据建模时将用户、入住人、酒店、房型、订单、订单明细、评价等对象拆分为独立实体，并通过主外键关系表达业务联系。

在完整性设计方面，系统使用主键、外键、唯一约束、默认值和 CHECK 约束控制基础数据质量；使用触发器维护房型可预订数量，避免订单明细与库存不一致；使用存储过程实现用户订单查询、入住率统计、预订情况统计和入住人画像统计。

在物理设计方面，系统围绕酒店检索、订单查询和统计报表建立索引，并通过视图复用多表连接结果。通过本次设计，我进一步理解了数据库设计不仅是建表，还包括业务约束、数据一致性、查询性能、安全权限和运行测试等内容。该数据库能够支撑一个前后端分离的酒店预订系统，也能作为课程答辩中说明数据库设计思想和实现过程的材料。